E-Commerce API

Project Document

ASP.NET Core | Clean Architecture | CQRS | DDD

Version 1.0 • 2025

1. Project Overview

Full e-commerce REST API built with ASP.NET Core following Clean Architecture, CQRS with MediatR, and Domain-Driven Design (DDD). The system handles the full shopping lifecycle: browsing products, managing a cart, placing orders, processing payments, and tracking delivery.

1.1 Tech Stack

Layer	Technology
Backend Framework	ASP.NET Core 8
Architecture	Clean Architecture + CQRS + DDD
ORM	Entity Framework Core
Database	SQL Server
Authentication	JWT Bearer Tokens
Mediator	MediatR
Validation	FluentValidation
Mapping	Mapster / AutoMapper

2. Domain Entities

The following entities make up the domain layer of the application:

Entity	Description
User	System user — Customer or Admin
Address	Delivery address linked to a user
Product	Item available for purchase
Category	Product classification
Cart	User's active shopping cart
CartItem	Single product line inside a cart
Order	Confirmed purchase created from a cart
OrderItem	Single product line inside an order
OrderStatusHistory	Log of all status changes for an order
Payment	Payment record linked to an order
Coupon	Discount code — SingleUse or MultiUse
Review	User rating and comment on a product
Notification	System notification sent to a user

3. Enums

Enum	Values
OrderStatus	Pending, Confirmed, Shipped, Delivered, Cancelled
PaymentStatus	Pending, Succeeded, Failed, Refunded
PaymentProvider	Stripe, PayPal, Cash
CouponType	SingleUse, MultiUse
NotificationType	Order, Payment, System
UserRole	Customer, Admin

4. Project Structure

Project	Responsibility
ECommerce.Domain	Entities, Value Objects, Domain Events, Enums, Interfaces
ECommerce.Application	CQRS Commands/Queries, Handlers, DTOs, Validators, Interfaces
ECommerce.Infrastructure	EF Core DbContext, Configurations, Repositories, Services
ECommerce.API	Controllers, Middleware, JWT Setup, DI Registration

5. Features & Use Cases

5.1 Authentication

Use Case	Type	Endpoint
Register	Command	POST /api/auth/register
Login	Command	POST /api/auth/login
Refresh Token	Command	POST /api/auth/refresh

5.2 User Management

Use Case	Type	Endpoint
Get My Profile	Query	GET /api/users/me
Update Profile	Command	PUT /api/users/me
Add Address	Command	POST /api/users/me/addresses
Get My Addresses	Query	GET /api/users/me/addresses
Delete Address	Command	DELETE /api/users/me/addresses/{id}
Get All Users (Admin)	Query	GET /api/users
Delete User (Admin)	Command	DELETE /api/users/{id}

5.3 Product Management

Use Case	Type	Endpoint
Get All Products	Query	GET /api/products
Get Product By Id	Query	GET /api/products/{id}
Get Products By Category	Query	GET /api/products?categoryId={id}
Search Products	Query	GET /api/products?search={term}
Create Product (Admin)	Command	POST /api/products
Update Product (Admin)	Command	PUT /api/products/{id}
Delete Product (Admin)	Command	DELETE /api/products/{id}
Reduce Stock	Command	Internal — called on Order placed
Increase Stock	Command	Internal — called on Order cancelled

5.4 Category Management

Use Case	Type	Endpoint
Get All Categories	Query	GET /api/categories
Create Category (Admin)	Command	POST /api/categories
Update Category (Admin)	Command	PUT /api/categories/{id}
Delete Category (Admin)	Command	DELETE /api/categories/{id}

5.5 Cart Management

Use Case	Type	Endpoint
Get My Cart	Query	GET /api/cart
Add Item to Cart	Command	POST /api/cart/items
Update Item Quantity	Command	PUT /api/cart/items/{productId}
Remove Item from Cart	Command	DELETE /api/cart/items/{productId}
Clear Cart	Command	DELETE /api/cart

5.6 Order Management

Use Case	Type	Endpoint
Place Order (from Cart)	Command	POST /api/orders
Get My Orders	Query	GET /api/orders
Get Order By Id	Query	GET /api/orders/{id}
Get Order Tracking	Query	GET /api/orders/{id}/tracking
Cancel Order	Command	POST /api/orders/{id}/cancel
Apply Coupon to Order	Command	POST /api/orders/{id}/coupon
Update Order Status (Admin)	Command	PUT /api/orders/{id}/status
Get All Orders (Admin)	Query	GET /api/admin/orders

5.7 Payment

Use Case	Type	Endpoint
Pay for Order	Command	POST /api/payments
Get Payment by Order	Query	GET /api/payments/order/{orderId}

Use Case	Type	Endpoint
Webhook — Provider Callback	Command	POST /api/payments/webhook

5.8 Coupon Management

Use Case	Type	Endpoint
Validate Coupon	Query	GET /api/coupons/{code}/validate
Create Coupon (Admin)	Command	POST /api/coupons
Update Coupon (Admin)	Command	PUT /api/coupons/{id}
Delete Coupon (Admin)	Command	DELETE /api/coupons/{id}
Get All Coupons (Admin)	Query	GET /api/coupons

5.9 Reviews

Use Case	Type	Endpoint
Add Review	Command	POST /api/reviews
Get Product Reviews	Query	GET /api/products/{id}/reviews
Delete Review (Admin)	Command	DELETE /api/reviews/{id}

5.10 Notifications

Use Case	Type	Endpoint
Get My Notifications	Query	GET /api/notifications
Mark as Read	Command	PUT /api/notifications/{id}/read
Mark All as Read	Command	PUT /api/notifications/read-all

6. Domain Events

Domain Events are fired automatically when important actions happen inside the domain. They trigger side-effects like sending notifications, reducing stock, and updating totals.

Event	Triggered By	Side Effects
OrderPlacedEvent	Place Order command	Reduce stock, Send notification, Clear cart
OrderConfirmedEvent	Admin confirms order	Send notification to user
OrderShippedEvent	Admin marks shipped	Send notification to user
OrderDeliveredEvent	Admin marks delivered	Send notification to user
OrderCancelledEvent	User or Admin cancels	Restore stock, Send notification
PaymentSucceededEvent	Payment webhook	Change order status to Confirmed
PaymentFailedEvent	Payment webhook	Keep order as Pending, Notify user
UserRegisteredEvent	Register command	Send welcome notification

7. Business Rules

7.1 Order Rules

- Order can only be cancelled if status is Pending or Confirmed
- Order status can only move forward — cannot go back
- Placing an order requires at least one item in the cart
- Stock must be sufficient for all items before placing order
- Coupon must be valid before applying to order

7.2 Coupon Rules

- SingleUse coupon: can only be used once — IsUsed becomes true after use
- MultiUse coupon: can be used until UsedCount reaches UsageLimit
- Expired coupons are always invalid regardless of type
- One coupon per order only

7.3 Review Rules

- User can only review a product they have actually purchased
- User can only leave one review per product
- IsVerifiedPurchase is set automatically by checking order history

7.4 Cart Rules

- Each user has exactly one active cart
- Adding the same product again increases quantity
- Cart is cleared automatically after order is placed

7.5 Payment Rules

- Payment is linked to a specific order
- Multiple payment attempts are allowed per order (One-to-many)
- Order status changes to Confirmed only after PaymentSucceeded

8. Implementation Checklist

Phase 1 — Domain Layer

Task	Status
Create all Entity classes with private setters	☐
Create Value Object: ShippingAddress (Owned Entity)	☐
Create all Enums	☐
Add Domain Methods to Entities (AddItem, ApplyCoupon, etc.)	☐
Add Domain Events	☐
Define Repository Interfaces	☐

Phase 2 — Application Layer

Task	Status
Setup MediatR + CQRS structure	☐
Create DTOs for all entities	☐
Implement Commands and Handlers	☐
Implement Queries and Handlers	☐
Add FluentValidation for all Commands	☐
Add Domain Event Handlers	☐

Phase 3 — Infrastructure Layer

Task	Status
Setup EF Core DbContext	☐
Configure all Entity mappings (Fluent API)	☐
Configure ShippingAddress as Owned Entity	☐
Implement all Repositories	☐
Add Migrations and seed data	☐
Implement Email / Notification Service	☐

Phase 4 — API Layer

Task	Status
Setup JWT Authentication	☐
Implement all Controllers	☐
Add Role-based Authorization (Admin / Customer)	☐
Add Global Exception Handling Middleware	☐
Add Pagination to list endpoints	☐
Document API with Swagger	☐

Phase 5 — Extra Features

Task	Status
Add Soft Delete for User and Product	☐
Add filtering and sorting to product list	☐
Add image upload for products	☐
Add Payment Webhook endpoint	☐
Add Rate Limiting	☐
Write Unit Tests for Domain logic	☐

9. API Response Standard

All API endpoints return a unified response wrapper:

Field	Type	Description
isSuccess	bool	true if operation succeeded
statusCode	int	HTTP status code
message	string	Human-readable message
data	T?	Response payload (null on error)
errors	List<string>?	Validation or domain errors